

OOP

programare orientată pe obiecte

CURS03: RELATII INTRE CLASE

ASOCIERE. AGREGARE. COMPUNERE. MOSTENIRE.

Alexandru Olteanu

{alexandru.olteanu@upb.ro}

Computer Science & Engineering Department

FACULTATEA DE AUTOMATICĂ ŞI CALCULATOARE

Universitatea Politehnica Bucureşti

UNIFIED MODELING LANGUAGE (UML)

- modalitate standard de vizualizare a felului in care arata un sistem software
- multiple tipuri de diagrame; noi vom folosi la curs:
  - Class Diagram (structure)
  - Sequence Diagram (behaviour)

CLASS DIAGRAM

- One box for each class, divided into three compartments: (1) the name of the class, (2) its attributes, (3) its operations that most obviously correspond to the methods of a class
- "+" public, "#" protected, "-" private, "~" default (package)
- normally, you don't show those operations that simply manipulate properties, because they can usually be inferred .

-- Martin Fowler, UML Distilled

Class Diagram - exemplu:

```
class Point2D {
  private int x;
  private int y;

  public void setX(int x) {
    this.x = x;
  }

  public int getX() {
    return x;
  }

  public void print() {
    System.out.println("("+this.x+","+this.y+")")
  }
}
```

ASOCIEREA (ASSOCIATION)
AGREGAREA (AGGREGATION)
COMPUNEREA (COMPOSITION)

ASOCIERE

- relatie intre clase privite ca entitati independente
- cat de stransa este relatia? ganditi-va la durata de viata a celor doua entitati

ASOCIERE CARE NU ESTE AGREGARE:

- doua obiecte indeplinesc un scop comun
- is-using-a (Doctor D treats Pacient P)
- durata de viata separata, pot fi recombinate

```
class Doctor {
  public void treats (Pacient p) {
    ...
  }
}

class Pacient {
  public void is_treated_by (Doctor d) {
    ...
  }
}
```

AGREGARE

- gruparea a mai multe modele intr-o structura comuna
- has-a (Team T has Player P)
- durata de viata separata, pot fi recombinate

```
class Player {
  ...
}

public class Team {
  public String name ;
  public List <Player > roster ;
  ...
  public addPlayer ( Player p) {
    roster.add(p) ;
  }
}
```

COMPUNERE

- strong association
- part-of (Point2D center is part of Circle c)
- aceeasi durata de viata (revenim la final)

```
public class Point2D {
   ...
}

public class Circle {
   public Point2D center;
   public Float radius;
}
```

MULTIPLICITY

- relatia poate fi one-to-one, one-to-many, many-to-many
- de regula, se noteaza pe diagrame:
  - 1 (An order must have exactly one customer)
  - 0..1 (A corporate customer may or may not have a single sales rep.)
  - * (A customer need not place an Order and there is no upper limit to the number of Orders a Customer may place-zero or more orders.)

MOSTENIREA (INHERITANCE)
UPCAST, DOWNCAST
SUPRASCRIERE (OVERRIDE)

MOSTENIREA

- nu este asociere, este generalizare / specializare
- is-a (Doctor D is a Person P)
- O subclasa mosteneste toate câmpurile si metodele public si protected ale superclasei

```
public class Person {                       // super-clasa
  public String name;
  ...
}

public class Doctor extends Person {    // sub-clasa
  public Specialisation spec;
  ...
}

public class Patient extends Person {   // sub-clasa
  public Diagnosis diagnosis;
}
```

SUPRASCRIERE (OVERRIDE)

- O clasa poate suprascrie orice metoda dintr-o superclasa prin definirea unei metode cu aceeasi semnatura (inclusiv constructori).

```
public class Person {
  public void train (Gym gym) {
    gym.liftWeights () ;
  }
}

public class Sportsperson extends Person {
  public void train (Gym gym) {
    gym.liftBIGWeights () ;
  }
}
```

SUPRASCRIERE (OVERRIDE)

- În metoda din subclasa se poate folosi cuvantul cheie super pentru a apela metoda suprascrisa din superclasa.

```
public class Person {
  public void train (Gym gym) {
    gym.enter();                          // atentie la situatii pot
    gym.liftWeights () ;
    gym.exit();
  }
}

public class Sportsperson extends Person {
  public void train (Gym gym) {
    for (int i=0; i<10; i++)
      super.train (gym) ;
  }
}
```

SUPRASCRIERE VS. CONSTRUCTORI

- Urmatorul cod compileaza sau nu?

```
public class Person {
  protected String name ;
  public Person( String name ) {
    this.name = name ;
  }
}
public class Coach extends Person {
}
```

SUPRASCRIERE VS. TOSTRING ETC.

- Implicit, orice clasa mosteneste de la ***java.lang.Object*** metodele ***clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait*** si ar trebui sa le suprascrie cu ceva relevant pentru clasa respectiva

```
public class Person {
  protected String name ;
  public Person( String name ) {
    this.name = name ;
  }
  public String toString () {
    return name ;
  }
}
```

UPCAST VS DOWNCAST

```
public class Person {
  protected String name ;
  public String getName () {
      return name ;
  }
  public void greet (Person otherPerson ) {
      System.out.println ("Hi "+ otherPerson.getName () ) ;
  }
}

public class Doctor extends Person {
  public String getName () {
    return " dr "+ name ;
  }
  public void treat(Person p) {
    ...
```

UPCAST

Casting to a supertype: se întâmpla automat.

```
...

Person pete = new Person (" Pete ") ;
Person dave = new Doctor (" Dave ") ;  // automat
pete.greet ( dave ) ;
```

UPCAST

Referinta de tip Person
la un obiect de tip Doctor la run-time

```
...

Person pete = new Person (" Pete ") ;
Person dave = new Doctor (" Dave ") ;
pete.greet ( dave ) ;                    // ce se afiseaza?
```

DOWNCAST

Trebuie facut explicit, doar atunci cand suntem siguri de tipul de la run-time

```
public class Person {

    public void askForHelp(Person someone) {
        if (someone instanceof Doctor)
            ((Doctor)someone).treat(this);
    }

}
```

```
public class Person {

    public void askForHelp(Person someone) {
        // if run-time type not checked: ClassCastException
        ((Doctor)someone).treat(this);
    }

}
```

Relatii intre clase (agregare, mostenire, compunere)

ale

DOWNCAST

Sau, poate chiar mai elegant, ne putem folosi de supraincarcare

```
public class Person {

    public void askForHelp(Doctor doctor) {
        doctor.treat(this);
    }

    public void askForHelp(Person someone) {
        someone.callDoctor(this);
    }

}
```

INHERITANCE BREAKS ENCAPSULATION

But class inheritance has some disadvantages, too. First, you can't change the implementations inherited from parent classes at run-time, because inheritance is defined at compile-time. Second, and generally worse, parent classes often define at least part of their subclasses' physical representation. Because inheritance exposes a subclass to details of its parent's implementation, it's often said that "inheritance breaks encapsulation"

Object composition is defined dynamically at run-time through objects acquiring references to other objects. Composition requires objects to respect each others' interfaces, which in turn requires carefully designed interfaces that don't stop you from using one object with many others. But there is a payoff. Because objects are accessed solely through their interfaces, we don't break encapsulation. Any object can be replaced at runtime by another as long as it has the same type. Moreover, because an object's implementation will be written in terms of object interfaces, there are substantially fewer implementation dependencies.

That leads us to our second principle of object-oriented design:
Favor object composition over class inheritance

-- Gang of Four, Design Patterns

-- Martin Fowler, UML Distilled

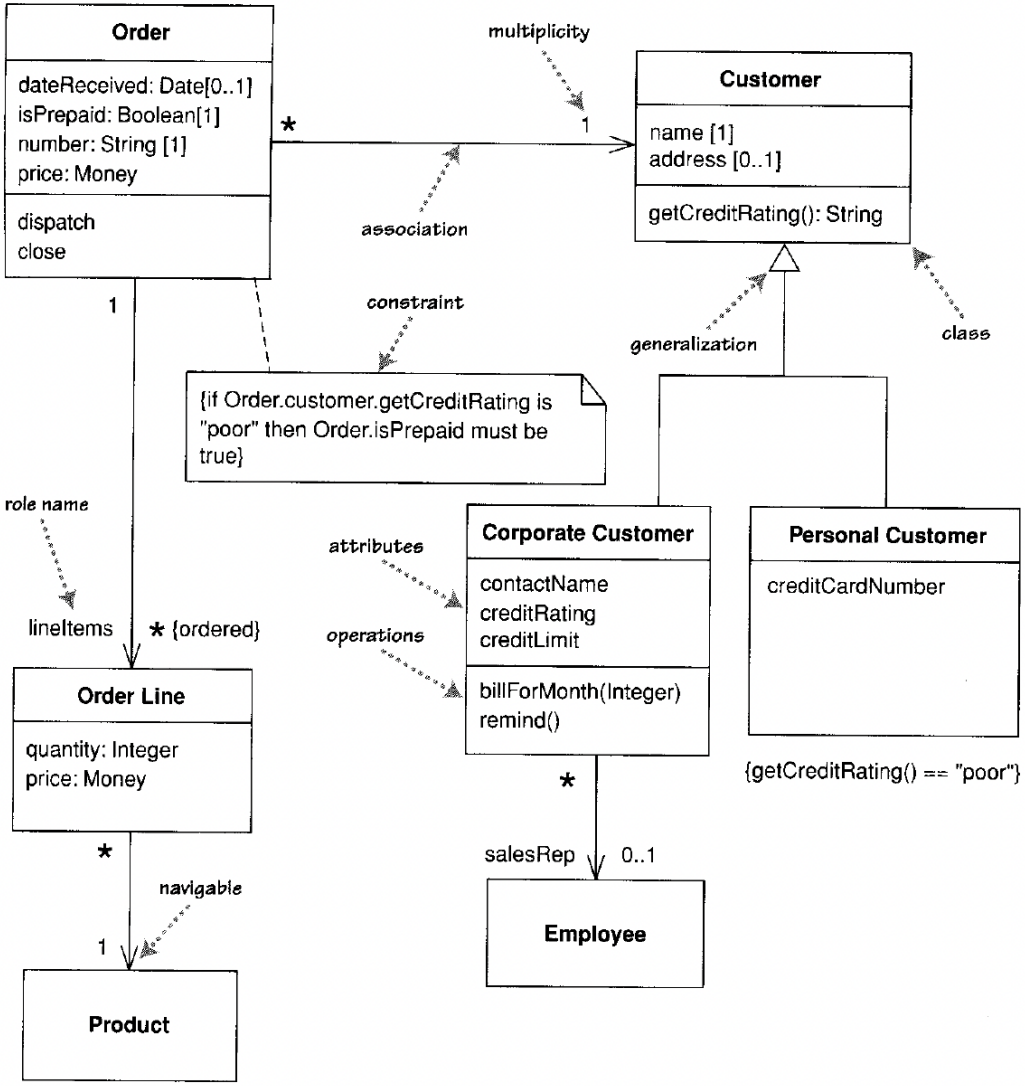


Figure 3.1 *A simple class diagram*

ndru.olteanu@upb.ro